

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчет по лабораторной работе №3
по дисциплине
«Объектно-ориентированное программирование»

Выполнил студент гр. ИВТб-2301-05-00	_____ /Макаров С.А./
Руководитель преподаватель	_____ /Шмакова Н.А./

Киров 2026

Цель работы

Цель работы: изучить и освоить принципы объектно-ориентированного программирования, включая инкапсуляцию, наследование и полиморфизм, путем создания иерархии классов в выбранной предметной области, взаимодействие с базой данных и графическим интерфейсом.

Задание

На основании согласованной в первой и второй лабораторной работе предметной области разработать приложение , используя классы и особенности java. Сведения об объектах предметной области необходимо хранить в БД.

Решение

Предметная область описывает управление ассортиментом блюд в заведении общественного питания. Система позволяет хранить информацию о различных блюдах, учитывать их специфические характеристики, рассчитывать итоговую цену для клиента в зависимости от этих характеристик и выводить информацию о блюде в удобном виде.

Программа содержит классы «Dish», представляющий базовый класс блюда, «Pizza», наследованный от «Dish» и представляющий класс пиццы, «Salad» наследованный от «Dish» и представляющий класс салата. Также для создания заказов существуют классы «Order» и «OrderItem», представляющий блюдо в заказе.

Базовый класс «Dish» содержит:

- Поля:
 - private UUID id – поле, означающее уникальный идентификатор;
 - private String name – поле, означающее название блюда, строковый тип данных;
 - private int weight – поле, означающее вес блюда, целочисленный тип данных;

- `private double price` – поле, цена блюда, формат с плавающей запятой;
- Конструкторы:
 - `public Dish(String name, int weight, double price)` - публичный конструктор, принимает в качестве параметров название, вес и цену блюда;
 - `public Dish(String name)` - публичный конструктор, принимает название блюда в качестве параметра;
 - `protected Dish()` – конструктор, необходимый для Hibernate;
- Абстрактные методы:
 - `public abstract String displayInfo()` – публичный абстрактный метод, возвращающий информацию о блюде, не принимает параметров;
 - `public abstract double calcFullPrice()` – публичный абстрактный метод, возвращающий полную цену блюда;
 - `public abstract DishResponse getInfo()` – публичный абстрактный метод, возвращающий информацию о блюде в необходимом формате для ответа на запрос к серверу.

Класс «Pizza», наследованный от класса «Dish» содержит:

- Собственные поля:
 - `private Dough dough = Dough.THICK;` – поле, означающее типа теста, по умолчанию THICK;
- Конструкторы:
 - `public Pizza() super();` – публичный конструктор, необходимый для Hibernate;
 - `public Pizza(String name) super(name);` – публичный конструктор, принимает название пиццы в качестве параметра;

- `public Pizza(String name, int weight, double price) super(name, weight, price);` – публичный конструктор, принимает в качестве параметров название, вес и цену пиццы;
- Переопределенные методы:
 - `public String displayInfo()` – публичный метод, возвращающий информацию о пицце (название, тип теста, вес, цена), не принимает параметров, возвращает информацию в следующем виде:


```
Pizza: Margarita, thick dough, 450g, $18.00;
```
 - `public double calcFullPrice()` – публичный метод, возвращает полную цену в зависимости от типа теста (Thick: price * 1.5, Thin: price * 1.3), не принимает параметров;
 - `public DishResponse getInfo()` – публичный метод, возвращающий информацию о пицце для JSON формата;
- Собственные методы:
 - `public void changeDough()` – публичный метод, меняет тип теста, не принимает параметров;
 - `public String cutInSlices()` – публичный метод, нарезает пиццу на куски, не принимает параметров, возвращает информацию в следующем виде:

`Cutting the pizza into slices...;`

Класс «Salad», наследованный от класса «Dish» содержит:

- Собственные поля:
 - `public Dressing Dressing { get; set; } = Dressing.OliveOil` – свойство, означающее тип заправки, по умолчанию OliveOil;
- Конструкторы:
 - `public Salad() super();` – публичный конструктор, необходимый для Hibernate;

- `public Salad(String name) super(name);` – публичный конструктор, принимает название салата в качестве параметра;
- `public Salad(String name, int weight, double price) super(name, weight, price);` – публичный конструктор, принимает в качестве параметров название, вес и цену салата;
- Переопределенные методы:
 - `public String displayInfo()` – публичный метод, отображает информацию о салате (название, тип заправки, вес, цена), не принимает параметров, возвращает информацию в следующем виде:


```
Salad: Caesar, OliveOil dressing, 250g, $12.60;
```
 - `public double calcFullPrice()` – публичный метод, возвращает полную цену в зависимости от типа заправки (OliveOil: $\text{price} * 1.4$, Mayonnaise: $\text{price} * 1.3$), не принимает параметров;
 - `public DishResponse getInfo()` – публичный метод, возвращающий информацию о салате для JSON формата;
- Собственные методы:
 - `public void changeDressing()` – публичный метод, меняет тип заправки, не принимает параметров;
 - `public String tossWithDressing()` – публичный метод, перемешивает салат с заправкой, не принимает параметров, возвращает информацию в следующем виде:

`Tossing the salad with OliveOil...`;

Класс «Order» содержит:

- Поля:
 - `private UUID id` – поле, означающее уникальный идентификатор;
 - `private LocalDateTime createdAt = LocalDateTime.now()` – поле, означающее дату создания заказа, данные типа Дата;

- `private double cost` – поле, стоимость заказа, формат с плавающей запятой;
- `private List<OrderItem> items = new ArrayList<>()` – поле, содержащее список блюд в заказе.

Класс «OrderItem» содержит:

- Поля:
 - `private UUID id` – поле, означающее уникальный идентификатор;
 - `private Order order = LocalDateTime.now()` – поле, означающее заказ, к которому относится блюдо;
 - `private Dish dish` – поле, означающее блюдо, доступное из меню;
 - `private int quantity` – поле, означающее количество блюда.

Для описания моделей Hibernate используются следующие нотации:

- `@Entity` – используется для описания сущности:

```
@Entity
public abstract class Dish { ... }
```

- `@Table(name = "dishes")` – используется для задания имени таблицы сущности в базе данных:

```
@Table(name = "dishes")
public abstract class Dish { ... }
```

- `@Inheritance(strategy = InheritanceType.JOINED)` – задает стратегию наследования. Общие поля родительского класса в одной таблице, а специальные поля наследников в отдельных таблицах, связанных по идентификатору:

```
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Dish { ... }
```

- @PrimaryKeyJoinColumn(name = "dish_id") – используется в наследниках и показывает, что их первичный ключ одновременно является внешним ключом на запись в родительской таблице:

```
@PrimaryKeyJoinColumn(name = "dish_id")
public class Pizza extends Dish { ... }
```

- @OnDelete(action = OnDeleteAction.CASCADE) – задает правила удаления записей в связанных таблицах:

```
@OnDelete(action = OnDeleteAction.CASCADE)
public class Pizza extends Dish { ... }
```

- @Id и @GeneratedValue – используются для создания уникального сгенерированного идентификатора:

```
@Id
@GeneratedValue
private UUID id;
```

- @ManyToOne() и @OneToMany(mappedBy = "order cascade = CascadeType.ALL, orphanRemoval = true) – создают связь один ко многим между сущностями:

```
public class Order {
    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<OrderItem> items = new ArrayList<>();
}
public class OrderItem {
    @ManyToOne()
    private Order order;
}
```

- @JoinColumn(name = "order_id") – используется для указания по какому полю связывать сущности:

```
public class OrderItem {
    @JoinColumn(name = "order_id")
```

```

    private Order order;
}

```

Диаграмма классов сущностей представлена на рисунке 1.

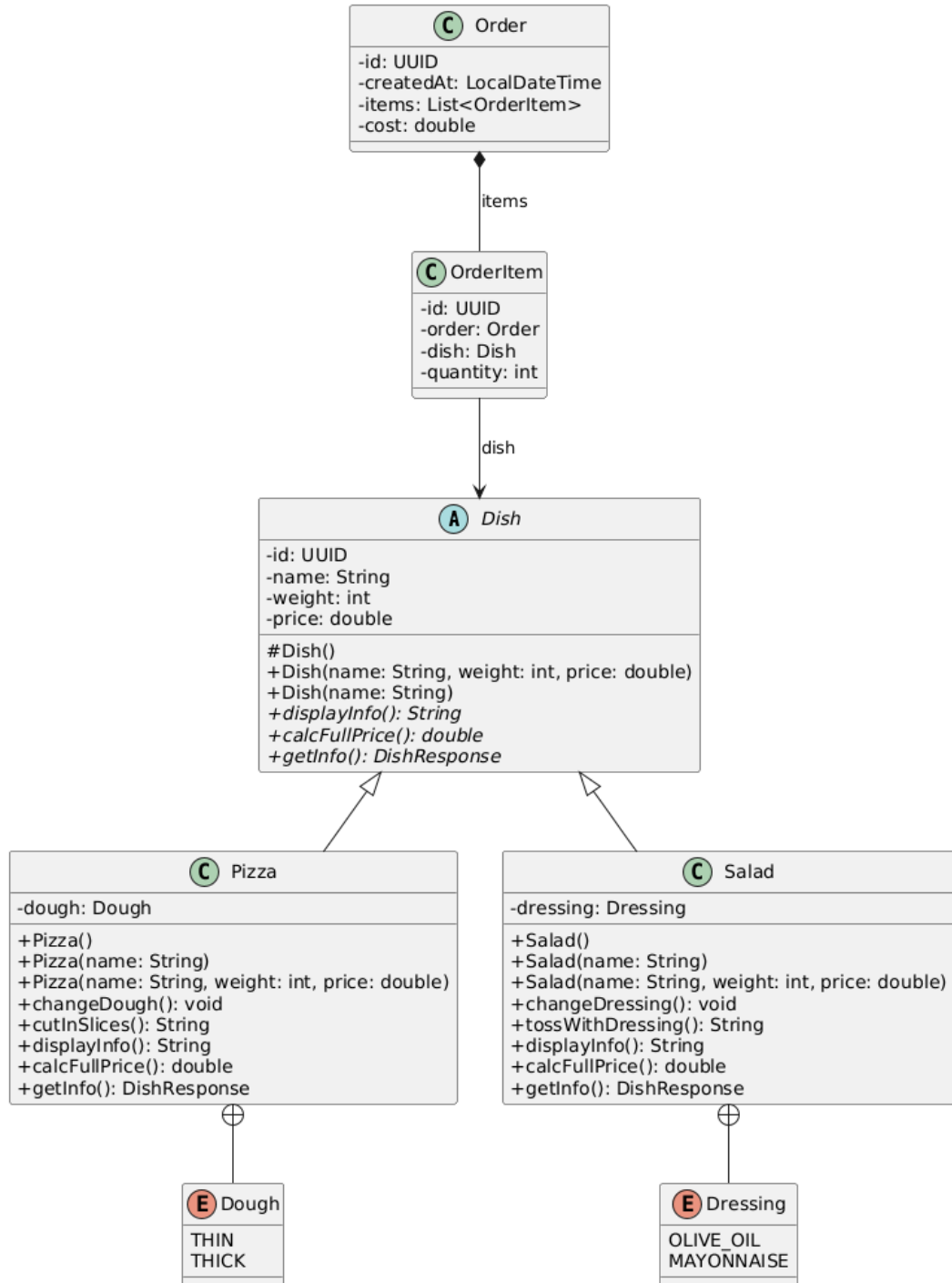


Рисунок 1 – Диаграмма классов сущностей

Разработаны серверные и клиентское приложение в виде веб-интерфейса. Серверное приложение содержит подключение к базе данных, конфигурацию моделей и контроллеры.

Подключение к базе данных Postgres происходит с помощью следующих настроек подключения:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/restaurant_db
spring.datasource.username=root
spring.datasource.password=root
```

Для того, чтобы пользоваться CRUD функциями у сущностей необходимо создать репозиторий. Например, для сущности «Pizza» репозиторий выглядит следующим образом:

```
public interface PizzaRepository
    extends JpaRepository<Pizza, UUID> {}
```

Репозитории для остальных сущностей выполняются аналогичным способом и содержат аналогичные методы.

Следующим шагом идет создание сервисов для управления данными, получаемых из репозитория. Например, «PizzaService» содержит следующие методы:

- public Pizza createPizza(Pizza pizza) – метод, который создает новую запись о пицце:

```
return pizzaRepository.save(pizza);
```

- public List<Pizza> getAllPizzas() – метод, возвращающий весь список пицц, хранящихся в базе данных:

```
return pizzaRepository.findAll();
```

- public Pizza getPizzaById(UUID id) – метод, возвращающий отдельную запись о пицце по уникальному идентификатору:

```
return pizzaRepository.findById(id);
```

- `public Pizza updatePizza(UUID id, Pizza pizzaDetails)` – метод, который обновляет записи о пицце по уникальному идентификатору:

```
Pizza existingPizza = getPizzaById(id);
existingPizza.setName(pizzaDetails.getName());
existingPizza.setWeight(pizzaDetails.getWeight());
existingPizza.setPrice(pizzaDetails.getPrice());
existingPizza.setDough(pizzaDetails.getDough());
return pizzaRepository.save(existingPizza);
```

- `public void deletePizza(UUID id)` – метод, удаляющий запись о пицце по уникальному идентификатору:

```
pizzaRepository.deleteById(id);
```

- `public double calculatePizzaPrice(UUID id)` – метод, возвращающий полную стоимость пиццы по уникальному идентификатору;
- `public Pizza toggleDough(UUID id)` – метод, меняющий тип теста пиццы по уникальному идентификатору;
- `public String getPizzaDisplayInfo(UUID id)` – метод, возвращающий полное описание записи пиццы по уникальному идентификатору;
- `public String cutPizza(UUID id)` – метод, возвращающий сообщение о нарезании пиццы на кусочки по уникальному идентификатору.

Сервисы для остальных сущностей выполняются аналогичным способом и содержат аналогичные методы.

Далее необходимо создать контроллер, который отправляет ответы на запросы клиента, используя методы, содержащиеся в сервисах. Например, «`PizzaController`» содержит следующие методы:

- `public Pizza create(@RequestBody Pizza pizza)` – метод, который создает новую запись о пицце;
- `public List<Pizza> getAll()` – метод, возвращающий весь список пицц, хранящихся в базе данных;

- `public Pizza getOne(@PathVariable UUID id)` – метод, возвращающий отдельную запись о пицце по уникальному идентификатору;
- `public Pizza update(@PathVariable UUID id, @RequestBody Pizza pizzaDetails)` – метод, который обновляет записи о пицце по уникальному идентификатору;
- `public void delete(@PathVariable UUID id)` – метод, удаляющий запись о пицце по уникальному идентификатору;
- `public String cut(@PathVariable UUID id)` – метод, возвращающий сообщение о нарезании пиццы на кусочки по уникальному идентификатору;
- `ppublic String getInfo(@PathVariable UUID id)` – метод, возвращающий полное описание записи пиццы по уникальному идентификатору;
- `public double getPrice(@PathVariable UUID id)` – метод, возвращающий полную стоимость пиццы по уникальному идентификатору;
- `public Pizza changeDough(@PathVariable UUID id)` – метод, меняющий тип теста пиццы по уникальному идентификатору.

Контроллеры для остальных сущностей выполняются аналогичным способом и содержат аналогичные методы.

Диаграмма классов репозитория, сервисов, контроллеров представлена на рисунке 2.

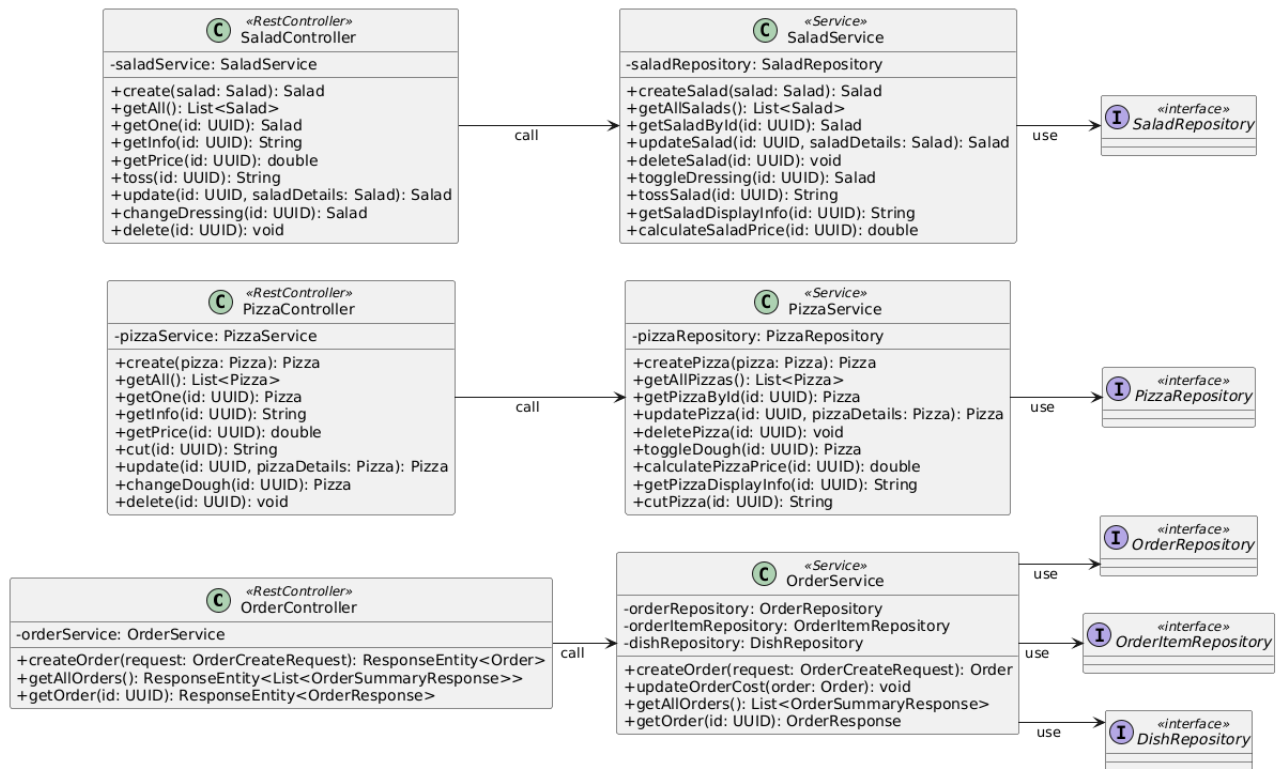


Рисунок 2 – Диаграмма классов репозитория, сервисов, контроллеров

Разработан веб-интерфейс для взаимодействия с WEB API, экранные формы которого представлены на рисунках 3, 4, 5, 6.

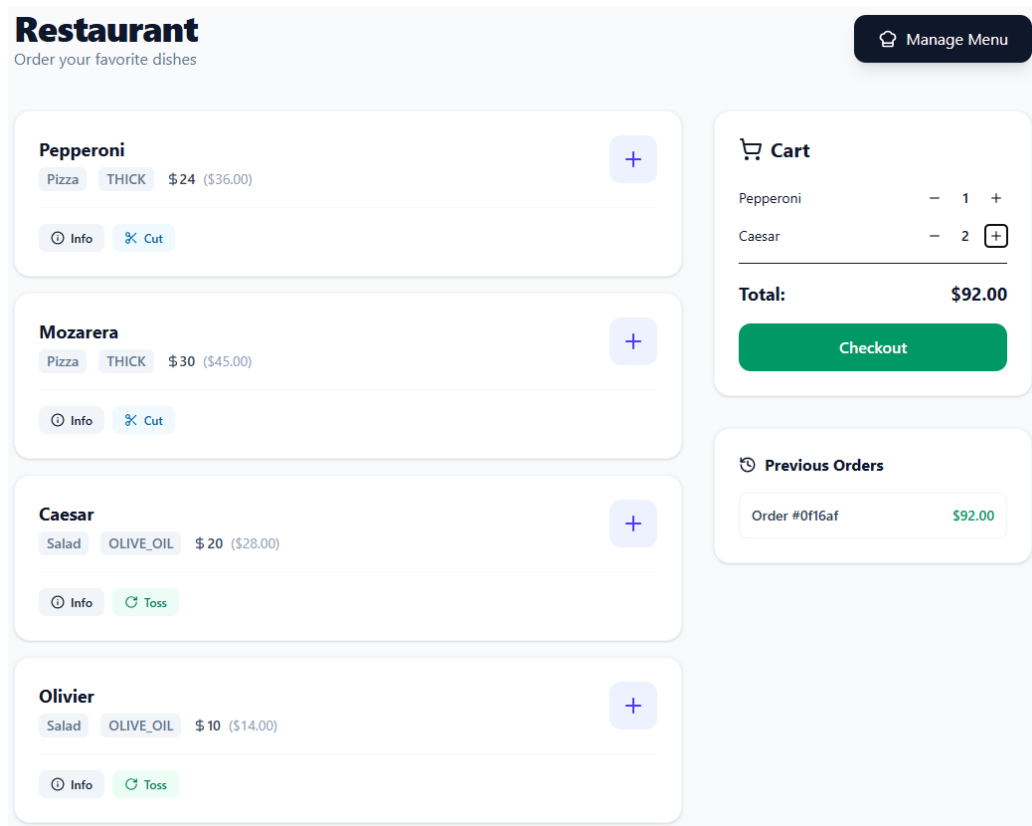


Рисунок 3 – Страница со списком блюд, корзиной, заказов

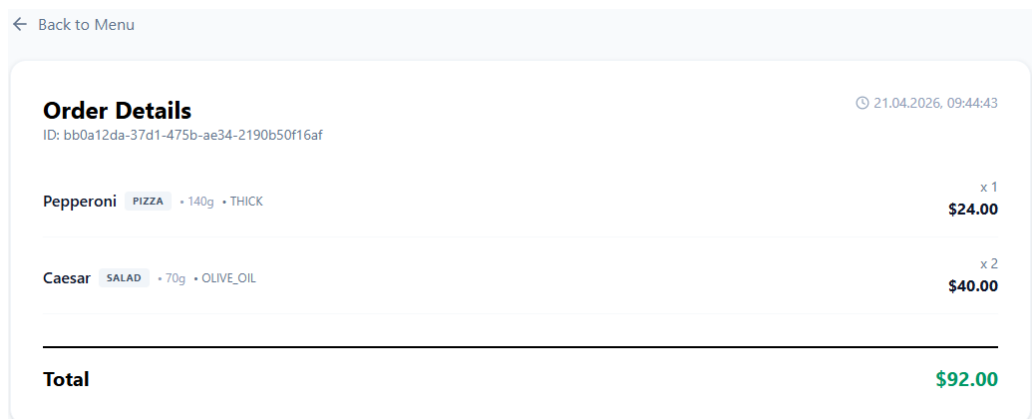


Рисунок 4 – Страница отдельного заказа

Menu Management

Pizza Salad

Pepperoni

Mozarera

+ Add New

Рисунок 5 – Форма со списком блюд

Edit Dish

Pepperoni

140 24

Switch to Thin Dough

Save Details Cancel

Рисунок 6 – Форма редактирования блюда

Вывод

В ходе выполнения лабораторной работы была разработана и реализована иерархия классов, моделирующая управление ассортиментом блюд, создание закалв в заведении общественного питания. Разработана база данных и веб-приложение, демонстрирующее работу системы.